

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV

COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO4: Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

Data Interpretation

Code of Conduct:

*I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: K. Pujitha

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	

Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	

Annexure A

Data Interpretation

Question 1: Hough Transform & Shape Detection

Scenario: Detect linear features (roads, rivers) in satellite images.

Input Data:

- A. High-resolution grayscale satellite images of urban and rural areas (e.g., 1024 × 1024 pixels, TIFF or PNG)
- B. Images contain noise (clouds, shadows) and partial occlusions
- C. Multiple linear features (roads, rivers, canals) with intersections
- D. Optional synthetic dataset: images with drawn lines at random angles and lengths to test Hough Transform

Question 2: PCA & Shape Priors for Object Recognition

Scenario: Segment organs in MRI scans using shape priors.

Input Data:

- A. MRI slices (grayscale, 512 × 512 pixels) of organs like liver or heart
- B. Training set: 50–100 annotated organ masks for PCA-based shape model construction
- C. Test set: 10–20 MRI images with partial organ boundaries or noise
- D. File formats: DICOM or PNG

Question 3: Machine Learning Methods (SVM, HMM, GMM, EM)

Scenario: Human activity recognition from video feature trajectories.

Input Data:

- A. Video sequences (30–60 FPS) of activities like walking, running, jumping (e.g., 640 × 480 pixels, MP4)
- B. Feature trajectories extracted per frame: joint positions (x, y coordinates) or optical flow vectors
- C. Training labels: activity per video (e.g., walking, jumping)
- D. Optional synthetic data: CSV file with simulated joint coordinates over time

Question 4: Deep Learning (CNNs, Transformers, GANs)

Scenario: Detect rare obstacles in urban self-driving car images.

Input Data:

- A. RGB camera images of urban streets (1024 × 768 pixels, JPG/PNG)
- B. Labeled dataset with bounding boxes for obstacles (rare objects, e.g., traffic cones, animals)
- C. Small dataset: 500–1000 labeled images for real obstacles
- D.

Optional synthetic images generated by GAN for data augmentation

Question 5: Graph-Based Learning for Shape Correspondence

Scenario: Find correspondences between 3D human face meshes.

Input Data:

- A. 3D meshes of human faces (OBJ or PLY format) with varying vertex counts (5,000–50,000 vertices)
- B. Training set: 20–50 face meshes with annotated landmark correspondences
- C. Test set: 5–10 unseen face meshes
- D. Optional: Graph adjacency matrices representing each mesh for input to graph-based algorithms

Question 1: Hough Transform & Shape Detection

Scenario

The objective is to detect linear features such as roads, rivers, and canals from satellite images using the Hough Transform.

Input Data

1. **High-resolution grayscale satellite images**
 - Image size: 1024×1024 pixels
 - Format: TIFF or PNG
 - Images contain urban and rural areas with roads, rivers, and canals.
2. **Image Noise**
 - Images may contain noise caused by:
 - Clouds
 - Shadows
 - Atmospheric disturbances
 - Some linear features may be partially hidden or unclear.
3. **Multiple Linear Features**
 - The images contain multiple roads, rivers, and canals.
 - Features may have different:
 - Angles
 - Lengths
 - Positions
 - Some features may intersect with each other.
4. **Optional Synthetic Dataset**
 - Images can contain artificially generated lines.
 - Lines can have random angles and lengths.
 - This dataset can be used to test the accuracy of the Hough Transform.

Methodology

The Hough Transform can be applied using the following steps:

Input Satellite Image → Grayscale Conversion → Noise Removal → Edge Detection → Hough Transform → Line Detection → Output Image

Step 1: Preprocessing

The satellite image is converted into grayscale if required. Noise caused by clouds and shadows can be reduced using a suitable filtering technique.

Step 2: Edge Detection

An edge detection method such as the Canny Edge Detector is applied to identify the boundaries of roads, rivers, and canals.

Step 3: Hough Transform

The Hough Transform is applied to the detected edge pixels to identify straight lines.

The equation used is:

$$\rho = x \cos \theta + y \sin \theta$$

Where:

- x, y = coordinates of an edge pixel
- ρ = perpendicular distance from the origin
- θ = angle of the line

The Hough accumulator stores possible values of ρ and θ . Peaks in the accumulator indicate the presence of strong linear features.

Step 4: Line Detection

Lines corresponding to roads, rivers, and canals are detected based on the peaks in the Hough accumulator.

Step 5: Visualization

The detected lines are drawn over the original satellite image to clearly show the identified linear features.

Analysis

The Hough Transform is useful for detecting linear structures with different orientations and lengths. It can detect multiple lines even when they intersect. However, clouds, shadows, noise, and partial occlusions may produce unwanted edges and false line detections.

The detection accuracy can be improved by applying proper preprocessing, selecting a suitable edge detection threshold, and choosing an appropriate Hough Transform threshold.

Expected Output

The output should display the original satellite image with the detected roads, rivers, and canals highlighted as straight lines.

The result can be represented as:

Input Image → Edge Image → Hough Transform → Detected Linear Features

Conclusion

The Hough Transform is an effective technique for detecting straight-line features in satellite images. It can identify roads, rivers, canals, and other linear structures even when they have different angles and lengths. Proper preprocessing and edge detection are important for reducing the effects of noise, clouds, shadows, and partial occlusions.

Question 2: PCA & Shape Priors for Object Recognition

Scenario

The objective is to segment organs such as the liver or heart from MRI scans using PCA-based shape priors. PCA is used to learn the common shape variation of an organ from annotated training masks and then use this shape information to segment organs in new MRI images.

Input Data

1. MRI Images

- Grayscale MRI slices of organs such as the liver or heart.

- Image size: 512×512 pixels.
- Images may contain noise and unclear boundaries.
- 2. Training Set**
 - Contains 50–100 annotated organ masks.
 - The masks represent the actual boundaries of the organs.
 - These masks are used to construct a **PCA-based statistical shape model**.
- 3. Test Set**
 - Contains 10–20 MRI images.
 - Some images have:
 - Partial organ boundaries
 - Noise
 - Weak or unclear edges
 - The learned shape prior helps identify the complete organ boundary.
- 4. File Formats**
 - **DICOM**
 - **PNG**

Methodology

The PCA-based shape prior can be applied using the following steps:

MRI Images → Preprocessing → Annotated Masks → Shape Alignment → PCA Model → Shape Prior → Test Image Segmentation → Final Organ Boundary

Step 1: Preprocessing

The MRI images are converted into a suitable grayscale format and noise is reduced using filtering techniques. Image normalization can also be performed to improve segmentation.

Step 2: Collect Training Masks

The annotated organ masks from the training set are collected. Each mask represents the shape and boundary of the organ.

Step 3: Shape Representation and Alignment

The organ boundaries are represented as coordinate points. The shapes are aligned to reduce differences caused by position, rotation, and scale.

Step 4: Apply PCA

Principal Component Analysis (PCA) is applied to the aligned training shapes.

The mean shape is calculated as:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

The shape variation can be represented as:

$$x = \bar{x} + Pb$$

Where:

- x = reconstructed organ shape
- $\bar{x}$ = mean organ shape
- P = principal component matrix
- b = shape parameters

PCA identifies the major variations in the organ shapes from the training data.

Step 5: Construct Shape Prior

The mean shape and principal components form a statistical shape model. This model acts as a shape prior and represents the expected shape of the organ.

Step 6: Segment Test Images

The PCA shape model is applied to the test MRI images. Even when the organ boundary is partially missing or affected by noise, the shape prior helps estimate the complete organ boundary.

Step 7: Generate Final Segmentation

The estimated shape is combined with image information such as edges and intensity differences to obtain the final organ segmentation.

Analysis

PCA helps capture the common shape characteristics and variations of organs from the training masks. It is particularly useful when MRI images contain incomplete or noisy boundaries.

For example, if a part of the liver boundary is unclear, the PCA shape model can use the learned liver shape to estimate the missing region.

However, PCA-based segmentation depends strongly on the quality and variety of the training masks. If the training dataset contains limited shape variations, the model may not perform well on unusual organ shapes.

Expected Output

The output should show the segmented organ boundary over the MRI image.

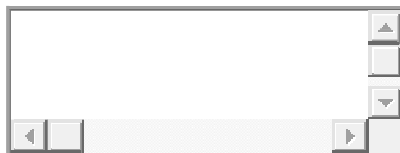
The result can be represented as:

MRI Image → Preprocessing → PCA Shape Model → Shape Prior →
Segmentation → Final Organ Mask

The final output should accurately identify the liver or heart region, including areas where the original boundary is partially unclear.

Conclusion

PCA-based shape priors provide an effective method for organ segmentation in MRI images. The PCA model learns the mean shape and major shape variations from annotated training masks. This learned shape information helps recover complete organ boundaries when test images contain noise or partial boundaries. The accuracy of the method depends on the quality and diversity of the training dataset.



Question 3: Machine Learning Methods (SVM, HMM, GMM, EM)

Scenario

The objective is to recognize human activities from video feature trajectories using machine learning methods such as SVM, HMM, GMM, and EM. The system classifies activities such as walking, running, and jumping based on the movement of body joints or optical flow over time.

Input Data

1. Video Sequences

- Videos contain human activities such as:
 - Walking
 - Running
 - Jumping
- Frame rate: 30–60 FPS
- Resolution: 640×480 pixels
- Format: MP4

2. Feature Trajectories

- Features are extracted from each video frame.
- Examples include:
 - Joint positions (x,y)
 - Optical flow vectors
- These features describe how the person moves over time.

3. Training Labels

- Each video is assigned an activity label.
- Examples:
 - Walking
 - Running
 - Jumping

4. Optional Synthetic Data

- A CSV file containing simulated joint coordinates over time can be used.
- This data can be used to test the machine learning algorithms.

Methodology

The human activity recognition system can be implemented using the following steps:

Video Input → Feature Extraction → Feature Preprocessing → Model Training → Activity Classification → Evaluation

Step 1: Video Preprocessing

The input video is divided into individual frames. The required motion information is extracted from consecutive frames.

Step 2: Feature Extraction

Feature trajectories are extracted from each frame.

For example, joint positions can be represented as:

$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Optical flow can also be used to represent the direction and magnitude of motion.

Step 3: Feature Preprocessing

The extracted features are normalized and arranged as time-series data. Missing or noisy feature values can be removed or corrected.

Step 4: SVM Classification

Support Vector Machine (SVM) can be used to classify activities based on extracted features.

For example:

- Walking → Class 1
- Running → Class 2
- Jumping → Class 3

SVM finds a decision boundary that separates the different activity classes.

Step 5: HMM Modeling

Hidden Markov Model (HMM) is useful because human activities occur as a sequence of movements over time.

For example:

Walking → Left-leg movement → Right-leg movement → Left-leg movement → ...

HMM models the transition between different hidden movement states and the observed feature trajectories.

Step 6: GMM Modeling

Gaussian Mixture Model (GMM) represents the distribution of feature vectors using multiple Gaussian distributions.

It can model different patterns of movement within an activity.

For example, different walking speeds can be represented by different Gaussian components.

Step 7: EM Algorithm

The Expectation-Maximization (EM) algorithm can be used to estimate the parameters of the GMM.

It works iteratively:

1. **Expectation (E-step):** Estimate the probability that each feature belongs to each Gaussian component.
2. **Maximization (M-step):** Update the parameters of the Gaussian distributions.
3. Repeat until the model converges.

Step 8: Activity Recognition

The trained models are applied to test video sequences. The extracted feature trajectory is compared with the learned activity patterns.

The system predicts the most likely activity:

Walking / Running / Jumping

Analysis

Each method has a different role in human activity recognition:

Method	Purpose
SVM	Classifies activities into different categories

Method	Purpose
HMM	Models the temporal sequence of human movements
GMM	Models the probability distribution of movement features
EM	Estimates GMM parameters iteratively

SVM is effective for classification, while HMM is particularly suitable for activities where the order of movements is important. GMM and EM can model variations in feature distributions and movement patterns.

Expected Output

The system should identify the activity performed in the input video.

Example:

Input Video → Feature Extraction → Machine Learning Model → Predicted Activity

Input Activity Predicted Activity

Walking	Walking
Running	Running
Jumping	Jumping

The performance can be evaluated using accuracy, precision, recall, and confusion matrix.

Conclusion

Machine learning methods such as SVM, HMM, GMM, and EM can be effectively used for human activity recognition from video feature trajectories. SVM provides activity classification, HMM captures the temporal sequence of movements, while GMM and EM model the distribution of movement features. Combining these methods can improve the recognition of activities such as walking, running, and jumping.

Question 4: Deep Learning (CNNs, Transformers, GANs)

Scenario

The objective is to **detect rare obstacles in urban street images captured by self-driving cars** using deep learning techniques such as **CNNs, Transformers, and GANs**. The system identifies rare objects such as traffic cones and animals from road images.

Input Data

1. RGB Camera Images

- Images are captured using cameras mounted on self-driving vehicles.
- Image size: **1024 × 768 pixels**
- Format: **JPG / PNG**
- Images contain urban roads, vehicles, pedestrians, buildings, and other objects.

2. Labeled Dataset

- Each image contains **bounding-box annotations** around obstacles.
- Rare obstacles include:
 - Traffic cones
 - Animals
 - Other unusual objects on the road

3. Small Dataset

- Contains approximately **500–1000 labeled images** of real obstacles.
- Since rare obstacles have fewer training examples, data augmentation is required.

4. Optional Synthetic Dataset

- Synthetic images can be generated using a **GAN (Generative Adversarial Network)**.
- These images can increase the number and variety of training samples.

Methodology

The obstacle detection system can be implemented using the following steps:

Input Image → Preprocessing → Data Augmentation → CNN/Transformer Model → Object Detection → Bounding Boxes → Evaluation

Step 1: Image Preprocessing

The input RGB images are resized and normalized before being given to the deep learning model. Image augmentation techniques such as rotation, flipping, cropping, and brightness changes can increase the training data.

Step 2: CNN-Based Feature Extraction

A Convolutional Neural Network (CNN) is used to extract important visual features from the images.

The CNN can learn features such as:

- Edges
- Shapes
- Textures
- Object patterns

These features help distinguish rare obstacles from the background.

Step 3: Transformer-Based Detection

A vision Transformer can be used to understand relationships between different regions of the image.

It can identify the position and appearance of obstacles even when they occur in complex urban scenes.

Step 4: GAN-Based Data Augmentation

Because only 500–1000 real labeled images are available, a GAN can be used to generate additional synthetic obstacle images.

A GAN consists mainly of:

- **Generator:** Creates synthetic images.
- **Discriminator:** Determines whether an image is real or synthetic.

The generated images can be added to the training dataset to improve model generalization.

Step 5: Object Detection

The trained CNN/Transformer model detects obstacles and produces bounding boxes around them.

For example:

Image → Traffic Cone → Bounding Box

Image → Animal → Bounding Box

Step 6: Model Evaluation

The detection performance can be evaluated using:

- **Precision**
- **Recall**
- **F1-score**
- **Intersection over Union (IoU)**
- **Mean Average Precision (mAP)**

IoU is calculated as:

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

A higher IoU indicates that the predicted bounding box overlaps well with the actual obstacle.

Analysis

CNNs are effective for extracting visual features from road images, while Transformers can capture relationships between different regions of an image. GANs are useful for generating additional synthetic samples when real examples of rare obstacles are limited.

The main challenge is the small and imbalanced dataset. Some obstacles may appear only a few times, making them difficult for the model to learn. Synthetic data augmentation, transfer learning, and suitable preprocessing can help improve detection performance.

Expected Output

The system should identify rare obstacles and draw bounding boxes around them.

Example:

Obstacle	Detection Result
Traffic cone	Detected with bounding box
Animal	Detected with bounding box
Other rare obstacle	Detected with bounding box

Output Flow:

RGB Image → CNN/Transformer → Obstacle Detection → Bounding Box + Class Label + Confidence Score

Conclusion

Deep learning techniques such as CNNs, Transformers, and GANs can be effectively used to detect rare obstacles in urban self-driving car images. CNNs extract important visual features, Transformers learn relationships between image regions, and GANs generate synthetic training samples to overcome the problem of limited real data. The final system can detect rare obstacles such as traffic cones and animals using bounding boxes and class labels.

Question 5: Graph-Based Learning for Shape Correspondence

Scenario

The objective is to find correspondences between 3D human face meshes using graph-based learning methods. The system identifies matching points or landmarks between different 3D face models, even when the meshes have different numbers of vertices.

Input Data

1. 3D Face Meshes

- 3D models of human faces.
- File formats: OBJ or PLY
- Each mesh contains approximately 5,000–50,000 vertices.
- Different face meshes may have different numbers of vertices.

2. Training Set

- Contains 20–50 face meshes.
- Annotated landmark correspondences are provided.
- Examples of landmarks include:
 - Eyes
 - Nose
 - Mouth
 - Chin

3. Test Set

- Contains 5–10 unseen face meshes.
- The trained model is used to find corresponding landmarks on these new meshes.

4. Optional Graph Data

- Graph adjacency matrices can be provided for each face mesh.
- They represent the connections between neighboring vertices.
- These graphs can be used as input to graph-based learning algorithms.

Methodology

The graph-based shape correspondence system can be implemented using the following steps:

3D Face Mesh → Graph Construction → Feature Extraction → Graph-Based Learning → Landmark Matching → Correspondence Mapping → Evaluation

Step 1: Load 3D Face Meshes

The OBJ or PLY face meshes are loaded. Each mesh contains vertices and connections between vertices.

A vertex can be represented as:

$v_i = (x_i, y_i, z_i)$

where **x**, **y**, and **z** represent the 3D coordinates.

Step 2: Convert Mesh into Graph

Each 3D face mesh is represented as a graph:

$$G=(V,E)$$

Where:

- **V** = vertices of the face mesh
- **E** = edges connecting neighboring vertices

The adjacency matrix represents which vertices are connected to each other.

Step 3: Feature Extraction

Features are extracted from the vertices and their local neighborhoods.

Possible features include:

- 3D coordinates
- Surface curvature
- Vertex normal
- Local geometric information

These features help distinguish different regions of the face.

Step 4: Graph-Based Learning

A Graph Neural Network (GNN) or graph convolution method can be trained using the annotated face meshes.

The model learns relationships between neighboring vertices and learns which points correspond to the same facial landmarks across different face meshes.

Step 5: Learn Landmark Correspondences

The annotated training data provides known landmark pairs.

For example:

Left Eye → Left Eye

Nose Tip → Nose Tip

Mouth Center → Mouth Center

The graph-based model learns these correspondence patterns from the training meshes.

Step 6: Test on Unseen Faces

The trained model is applied to the 5–10 unseen test meshes.

It predicts corresponding landmarks and matches points between the new face meshes.

Step 7: Evaluate Correspondence

The predicted landmark positions are compared with the ground-truth annotations.

A common measure is the Euclidean distance:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

A smaller distance indicates better correspondence accuracy.

Analysis

Graph-based learning is suitable for 3D face correspondence because a face mesh naturally contains vertices and connections between neighboring vertices. Unlike methods that treat the mesh as a simple image, graph-based methods can use the geometric relationships between neighboring points.

The major challenge is that different face meshes may contain different numbers of vertices and different facial shapes. Graph-based learning can learn local and global geometric patterns to identify corresponding landmarks despite these differences.

The quality and accuracy of the correspondence depend on the number of training meshes, quality of landmark annotations, and variation in the face shapes.

Expected Output

The system should identify corresponding points between different 3D face meshes.

Example:

Landmark	Reference Face	Test Face
Left Eye	Landmark 1	Corresponding point
Right Eye	Landmark 2	Corresponding point
Nose Tip	Landmark 3	Corresponding point
Mouth Center	Landmark 4	Corresponding point
Chin	Landmark 5	Corresponding point

Output Flow:

3D Face Mesh → Graph Representation → Graph Neural Network → Landmark Detection → Corresponding Points

Conclusion

Graph-based learning is an effective approach for finding shape correspondences between 3D human face meshes. The face mesh is represented as a graph containing vertices and edges, and the graph-based model learns geometric relationships between facial regions. Using annotated training meshes, the model can identify corresponding landmarks on unseen faces. The accuracy can be evaluated using landmark correspondence error or Euclidean distance.